1. Name the device where programs are temporarily stored while they are being executed? **a) RAM**
2. Select the primary functions of an operating system of computer – **b) Manage hardware resources**
3. CPU stands for – **b) Central Processing Unit**
4. Select the function of the control unit in a CPU? – **c) Decode instructions and control data flow**
5. The process of drawing a flowchart for an algorithm is called – **d) Flowcharting**
6. Actual instructions in flowcharting are represented in - **b) Boxes**
7. A detailed flowchart is called - **c) Micro**
8. In an algorithm, a decision symbol indicates: **c) A conditional statement**
9. ASCII value of "A" is - **b) 65**
10. ASCII value of "C" is - **d) 67**
11. 97 is ASCII value of - **c) "a"**
12. 32 is ASCII value of - **d) Space character**
13. 49 is ASCII value of - **a)"1"**
14. Which component of a computer system is responsible for executing instructions? **b) CPU**
15. Which component of a computer system stores the BIOS (Basic Input/Output System)? **c) ROM**
16. Identify the purpose of cache memory in a computer system - **a) Store frequently used data for quick access**
17. Why is RAM considered volatile memory? **b) Because its contents are lost when power is turned off.**
18. How does the operating system facilitate communication between hardware components and software applications? **b) By managing memory allocation and scheduling tasks**
19. Why is cache memory faster than main memory (RAM)?

34. **Define escape sequence characters.**

An escape sequence in programming is a special character combination that begins with a backslash(/) and is used to represent characters that cannot be typed directly or have special meaning.

Ex. /n (New line), /t (Tab), // (Prints /)

35. Define type casting.

Type casting in programming refers to converting one data type into another. It is used when we need to perform operations between different data types or when a specific data type is required. Type casting is of two types:

1. Implicit type casting (Automatic Conversion)
2. Explicit type casting (Manual Conversion)

36. Describe the basic features of an algorithm.

An algorithm is a step-by-step procedure or set of rules used to solve a problem or perform a task. The basic features of an algorithm include:

1. **Input** – The algorithm takes zero or more inputs to process.
2. **Output** – It produces at least one output (result).
3. **Definiteness** – Each step is clear and unambiguous.
4. **Finiteness** – The algorithm must have a finite number of steps.
5. **Effectiveness** – Each step must be simple enough to be executed in a finite amount of time.
6. **Generality** – It should be applicable to a set of similar problems, not just a single case.

37. State the disadvantages of machine language and assembly language.

Disadvantages of Machine Language:

1. **Difficult to Understand** – Machine language consists of binary code (0s and 1s), making it hard for humans to read and write.
2. **Error-Prone** – Debugging and writing programs in machine language is complex and prone to errors.
3. **Hardware Dependent** – Programs written in machine language are specific to a particular CPU and cannot be easily transferred to another type of processor.
4. **Difficult to Modify** – Any changes require rewriting large portions of the code.
5. **Time-Consuming** – Writing programs in machine language takes significant time and effort.

Disadvantages of Assembly Language:

1. **Complex Syntax** – Although simpler than machine language, assembly language still requires knowledge of low-level hardware operations.
2. **Hardware-Specific** – Like machine language, assembly language is tied to a specific processor architecture.
3. **Longer Development Time** – Writing and debugging assembly code is time-consuming compared to high-level languages.
4. **Limited Portability** – Programs written in assembly language must be rewritten for different processor architectures.
5. **Lower Productivity** – Since it is closer to hardware, developing large-scale applications in assembly is inefficient compared to high-level languages.

These limitations led to the development of high-level programming languages, which are more user-friendly and portable.

38. Define a program. How it is different from an algorithm?

Definition of a Program:

A **program** is a set of instructions written in a programming language that a computer can execute to perform a specific task. It is an implementation of an algorithm using a programming language like Python, Java, or C++.

Difference Between a Program and an Algorithm:

Feature	***Algorithm***	***Program***
Definition	A step-by-step procedure to solve a problem	A set of instructions written in a programming language for execution by a computer.
Form	Written in natural language or pseudocode	Written in a specific programming language.
Execution	Cannot be directly executed by a computer.	Can be executed by a computer.
Specificity	General approach applicable to multi problems.	Specific implementation for a given task
Portability	Independent of programming languages	Depends on the language and platform used.

In short, an **algorithm** is a blueprint, while a **program** is the working implementation of that blueprint.

39. Define a flow chart. Discuss the basic features of flowchart.

Definition of a Flowchart:

A **flowchart** is a graphical representation of an algorithm or process using symbols to depict the sequence of operations. It visually illustrates the flow of control and helps in understanding, designing, and analysing a program or system.

Basic Features of a Flowchart:

1. **Start and End Points** – Every flowchart begins with a **Start** symbol and ends with an **End** symbol.
2. **Flow of Control** – The flow of execution is represented using arrows connecting different steps.
3. **Use of Standard Symbols** – Flowcharts use predefined symbols, such as:
    - **Oval** for Start/End
    - **Parallelogram** for Input/Output
    - **Rectangle** for Processes (calculations, assignments, etc.)
    - **Diamond** for Decision making (e.g., Yes/No conditions)
4. **Sequential Flow** – The process follows a logical step-by-step sequence.
5. **Decision Making** – The flowchart includes decision symbols to represent branching (e.g., if-else conditions).
6. **Modularity** – Large processes can be broken into sub-processes to improve clarity and manageability.
7. **Clarity and Simplicity** – A well-structured flowchart makes it easy to understand complex logic.

40. Describe the four basic data types. Discuss the range of the values they represent.

The **four basic data types** in most programming languages are:

1. Integer (int)

- Represents **whole numbers** (positive, negative, or zero) without decimal points.
- Commonly used in counting, indexing, and loop control.
- **Range:** Depends on the system and language:
    - **8-bit**: -128 to 127
    - **16-bit**: -32,768 to 32,767
    - **32-bit**: -2,147,483,648 to 2,147,483,647

- **64-bit**: -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807

2. Floating Point (float or double)

- Represents **real numbers** (numbers with decimals).
- Used in scientific calculations, measurements, and precise computations.
- **Range:**
  - **float (32-bit):** $\pm 1.5 \times 10^{-45}$ to $\pm 3.4 \times 10^{38}$
  - **double (64-bit):** $\pm 5.0 \times 10^{-324}$ to $\pm 1.8 \times 10^{308}$

3. Character (char)

- Represents **a single character** (letters, digits, symbols).
- Stored using ASCII or Unicode encoding.
- **Range:**
  - **ASCII (1 byte):** 0 to 255 (includes letters, digits, and special characters).
  - **Unicode (2 bytes or more):** 0 to 65,535 (supports characters from multiple languages).

4. Boolean (bool)

- Represents **true or false** values (binary logic).
- Used in decision-making and conditions.
- **Range:**
  - **true (1) or false (0)**

41. List and describe three primary components of a computer system.

A **computer system** consists of three primary components:

1. Central Processing Unit (CPU) – *The Brain of the Computer*

- The **CPU** processes instructions, performs calculations, and manages data flow.
- It consists of:
  - **Arithmetic Logic Unit (ALU):** Performs mathematical and logical operations.
  - **Control Unit (CU):** Directs operations, manages instruction execution.
  - **Registers:** Small, fast memory locations for temporary data storage.

2. Memory (Storage) – *Stores Data and Instructions*

- **Primary Memory (RAM & ROM):**
    - **RAM (Random Access Memory):** Temporary storage for active programs and data.
    - **ROM (Read-Only Memory):** Stores firmware and essential instructions.
- **Secondary Storage (Hard Drives, SSDs, USBs):**
    - Long-term storage for operating systems, applications, and user data.

3. Input and Output Devices (I/O Devices) – *Facilitate Communication*

- **Input Devices:** Allow users to enter data (e.g., keyboard, mouse, scanner).
- **Output Devices:** Display or produce results (e.g., monitor, printer, speakers).

42. List and explain the primary symbols used in flowcharts.

Flowcharts use **standard symbols** to visually represent the flow of a process. The primary symbols include:

1. Terminal Symbol (Start/End) – Oval : Represents the **beginning** or **end** of a flowchart.

2. Process Symbol – Rectangle : Represents an **operation**, such as a calculation, assignment, or action.

3. Input/Output Symbol – Parallelogram : Used for **reading inputs** (e.g., user input) and **displaying outputs** (e.g., printing results).

4. Decision Symbol – Diamond : Represents a **decision-making process** (e.g., "Yes" or "No" conditions).

5. Connector Symbol – Circle : Used when a flowchart continues to another part of the diagram, avoiding complex overlapping lines.

43. Explain the use of ASCII.

ASCII (American Standard Code for Information Interchange) is used to represent text in computers and communication systems. It assigns unique numeric values to characters (e.g., letters, digits, and symbols) to enable text processing and data exchange.

44. Give a numerical example of using modulus operation %.

Example:

17 % 5 = 2

Explanation: When 17 is divided by 5, the quotient is 3 and the remainder is **2**. The modulus operator (%) returns the remainder.

45. Predict what will happen if modulus operator % is used between two float variables.

If the modulus operator % is used between two float variables, it will return the remainder of their division as a float.

Example:
10.5 % 3.2 → **Result:** 0.9

Python and some other languages support float modulus, but some may not.

46. Explain precedence and associativity of operators with suitable examples.

Precedence of Operators : Precedence determines the order in which operators are evaluated in an expression. Operators with higher precedence are executed first.

Example:
5 + 3 * 2 → **Result:** 5 + 6 = 11 (Multiplication * has higher precedence than addition +)

Associativity of Operators: When two operators have the same precedence, associativity defines the direction in which they are evaluated (left to right or right to left).

Example (Left-to-Right Associativity):

10 - 4 - 2 → **Result:** (10 - 4) - 2 = 4 (Subtraction - is left-to-right associative)

Example (Right-to-Left Associativity):

2 ** 3 ** 2 → **Result:** 2 ** (3 ** 2) = 2 ** 9 = 512 (Exponentiation ** is right-to-left associative)

47. With examples explain the use of escape sequence characters.

Escape sequences in C are special characters preceded by a backslash (\). They help in formatting output.

Examples in C:

Newline (\n) – Moves text to a new line.

```
#include <stdio.h>
int main() {
   printf("Hello\nWorld!");
   return 0;
}
```

Output:

Hello

World!

Tab (\t) – Inserts a tab space.

```
#include <stdio.h>
int main() {
   printf("Hello\tWorld!");
   return 0;
}
```

Output:

Hello	World!

Double Quote (\") – Allows double quotes inside a string.

```
#include <stdio.h>
int main() {
   printf("He said, \"Hello!\"");
   return 0;
}
```

Output:

He said, "Hello!"

48. Provide one example of each type of memory and describe their respective uses.

Stack Memory (Automatic Memory)

1. Used for function calls and local variables.
2. Allocated automatically and deallocated when the function exits.

Example:

```
#include <stdio.h>
void func() {
   int x = 10;  // Stored in stack memory
   printf("Stack Memory Value: %d\n", x);
}
int main() {
   func();
   return 0;
}
```

Use: Fast memory used for function execution and local variables.

Heap Memory (Dynamic Memory)

1. Used for dynamic memory allocation using malloc, calloc, realloc, and free.

2. Memory must be manually freed to avoid leaks.

Example:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
   int *ptr = (int *)malloc(sizeof(int));  // Allocated in heap
   *ptr = 20;
   printf("Heap Memory Value: %d\n", *ptr);
   free(ptr);  // Freeing heap memory
   return 0;
}
```

Use: Stores dynamically allocated data like arrays and linked lists.

Data Segment (Global/Static Memory): Stores global and static variables throughout program execution.

Example:

```
#include <stdio.h>
int global_var = 50;  // Stored in data segment
int main() {
   static int static_var = 30;  // Also in data segment
   printf("Data Segment Values: %d, %d\n", global_var, static_var);
   return 0;
}
```

Use: Stores global and static variables with program lifetime.

Code Segment (Text Segment): Stores compiled program instructions (machine code).

Example:

```
#include <stdio.h>
void function() { printf("Code Segment Example\n"); }
int main() {
   function();  // Function code stored in code segment
   return 0;
}
```

Use: Holds executable code of the program.

Register Memory: Small, fast memory inside the CPU for frequently used variables.

Example:

```
#include <stdio.h>
int main() {
   register int reg_var = 5;  // Stored in CPU register (if available)
   printf("Register Memory Value: %d\n", reg_var);
   return 0;
```

}
Use: Speeds up access to frequently used variables.

49. Define an unsigned integer constant. Discuss the significance of declaring a constant unsigned.

An **unsigned integer constant** is a numeric constant that only stores non-negative whole numbers (0 and positive values). It is declared by appending a U or u at the end of the constant.

Example:

```
#include <stdio.h>
int main() {
   unsigned int num = 100U;  // Unsigned integer constant
   printf("Unsigned Constant: %u\n", num);
   return 0;
}
```

Significance of Declaring a Constant as Unsigned

1. **Prevents Negative Values:** An unsigned constant ensures that only positive values are stored, preventing accidental negative assignments.
2. **Increases Range of Values:** Since unsigned integers do not store negative numbers, they can store larger positive values.

   Example:

1. int (signed 4 bytes) → Range: **-2,147,483,648 to 2,147,483,647**
2. unsigned int (4 bytes) → Range: **0 to 4,294,967,295**

3. **Memory Efficiency:** In situations where negative values are unnecessary (e.g., array indexing, object counts), using unsigned constants optimizes memory usage.
4. **Avoids Unexpected Behaviour:** Using an unsigned constant prevents issues like negative loop counters or unexpected arithmetic errors.
5. **Improves Code Readability and Intent:** Declaring a constant as unsigned explicitly indicates that the variable will not store negative values, making the code easier to understand.

50. Provide a brief explanation of the function of each component of a computer system.

A computer system consists of several key components, each performing a specific function to ensure smooth operation.

1. **Central Processing Unit (CPU)** – The **brain of the computer** that executes instructions. It consists of:

Arithmetic Logic Unit (ALU): Performs mathematical and logical operations.

Control Unit (CU): Directs the flow of data and controls hardware components.

2. **Memory (RAM & ROM)** – Stores data and instructions temporarily or permanently.

RAM (Random Access Memory): Volatile memory that holds active programs and data.

ROM (Read-Only Memory): Non-volatile memory that stores essential startup instructions (BIOS/firmware).

3. **Storage Devices** – Used for long-term data storage.

HDD (Hard Disk Drive) / SSD (Solid State Drive): Store operating systems, applications, and user files.

4. **Input Devices** – Allow users to provide data and commands.

Examples: Keyboard, mouse, touchscreen, scanner, microphone.

5. **Output Devices** – Display or produce results of processed data.

Examples: Monitor, printer, speakers.

6. **Motherboard** – The **main circuit board** that connects and allows communication between all components.

7. **Power Supply Unit (PSU)** – Converts electrical power to a suitable form for the computer's components.

8. **Networking Components** – Enable communication with other devices.

Examples: Network Interface Card (NIC), Wi-Fi adapter, modem, router.

Each component plays a crucial role in ensuring the smooth functioning of a computer system.

51. Discuss the importance of flowchart.

A **flowchart** is a diagram that visually represents the steps of an algorithm or process using symbols. It is important for several reasons:

1. **Easy Understanding of Logic**: Flowcharts provide a clear visual representation of the logic behind a program or process, making it easier to understand and follow.

2. **Efficient Problem Solving**: Helps in breaking down complex problems into smaller steps, making it easier to analyse and solve.

3. **Better Communication**: Flowcharts serve as a common reference for programmers, designers, and stakeholders, improving collaboration and communication.
4. **Error Detection and Debugging**: By visualizing the process, errors and inefficiencies in logic can be detected early, reducing debugging time.
5. **Documentation and Maintenance**: A flowchart acts as a useful reference for future modifications, helping in maintaining and updating programs efficiently.

52. Explain the difference between volatile and non-volatile memory in a computer system.

Feature	*Volatile Memory*	*Non-Volatile Memory*
Definition	Temporary memory that loses data when power is turned off	Permanent memory that retains data even without power.
Example	RAM (Random Access Memory)	ROM (Read-Only Memory), HDD, SSD, Flash Memory
Speed	Faster access time	Slower compared to volatile memory
Usage	Used for active processes, running applications, and temporary storage.	Used for long-term storage of files, OS, and firmware.
Data Retention	Loses data when power is off.	Retains data even after power is turned off.

54. Discuss different C tokens.

In C programming, **tokens** are the smallest meaningful units of a program. The **different types of tokens** in C are:

1. **Keywords**: Reserved words in C that have predefined meanings.

   Examples: int, float, if, else, return, while, void

2. **Identifiers**: Names used for variables, functions, arrays, etc.

   Rules: Must start with a letter or _, followed by letters, digits, or underscores.

   Example: int age; float salary;

3. **Constants**: Fixed values that do not change during program execution.

   Types: Integer constants (100), Floating-point constants (3.14), Character constants ('A'), String constants ("Hello").

   Example: const int max = 50;

4. **Operators**: Symbols used for operations on variables and values.

Types:

1. Arithmetic (+, -, *, /, %)
2. Relational (==, !=, <, >, <=, >=)
3. Logical (&&, ||, !)

Example: int sum = 10 + 5;

5. **Special Symbols:** Include punctuation and special characters used in C syntax.

Examples: { } (Braces), [ ] (Brackets), ( ) (Parentheses), ; (Semicolon), , (Comma), # (Preprocessor directive).

Example: printf("Hello, World!");

6. **Strings & Punctuation**: Strings are sequences of characters enclosed in double quotes.

Example: "Welcome to C programming"

54. Explain implicit and explicit type conversions with example.

In C, type conversion refers to converting one data type into another. It can be implicit (automatic) or explicit (manual, type casting).

1. Implicit Type Conversion (Type Promotion)

1. Also called **automatic type conversion**.
2. Happens when a smaller data type is converted into a larger data type automatically.
3. Follows type hierarchy:

   char → int → float → double

Example:

```
#include <stdio.h>
int main() {
   int num = 10;
   float result = num + 5.5; // int is implicitly converted to float
   printf("Result: %f\n", result);
   return 0;
}
```

Output:

Result: 15.500000

Explanation: The int value (10) is automatically converted to float (10.0) before addition.

2. Explicit Type Conversion (Type Casting)

1. Also called manual type conversion.
2. Uses type casting to forcefully change the data type.
3. Syntax: (datatype) value

Example:

```
#include <stdio.h>
int main() {
   float num = 10.75;
   int result = (int) num; // Explicitly converting float to int
   printf("Result: %d\n", result);
   return 0;
}
```

Output:

Result: 10

Explanation: The float value 10.75 is explicitly converted to int, truncating the decimal part.

55. Design a flowchart to find the sum of first n prime numbers, where n should be given by the user.